

Programmazione 1

Corso c

09/10/2023

>while e for: il loop in C

Alessandro Mazzei

alessandro.mazzei@unito.it

Dipartimento di Informatica

Università degli Studi di Torino



UNIVERSITÀ
DI TORINO

Outline

- Il costrutto while
- Esercizi
- Variabili sentinella e contatore
- Il costrutto for
- Confronto for/while

Outline

- Il costrutto while
- Esercizi
- Variabili sentinella e contatore
- Il costrutto for
- Confronto for/while

Il costrutto while

- Il *costrutto di iterazione* (ciclo)

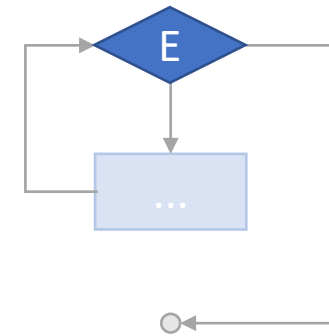
```
while (<espressione E>) {
```

...

```
}
```

...

valuta l'espressione E:



Il costrutto while

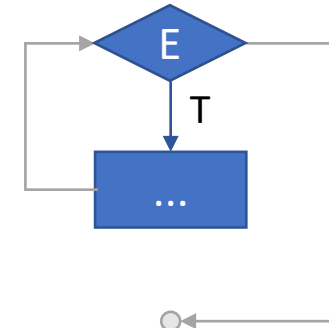
- Il *costrutto di iterazione* (ciclo)

```
while (<espressione E>) {  
    ...  
}  
...
```



valuta l'espressione E:

- se E è vera, il blocco viene eseguito ...



Il costrutto while

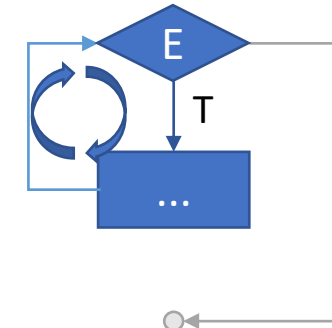
- Il *costrutto di iterazione* (ciclo)

```
while (<espressione E>) {  
    ...  
}  
...
```



valuta l'espressione E:

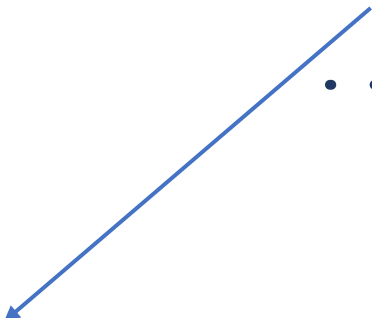
- se E è vera, il blocco viene eseguito ed il controllo torna al while()



Il costrutto while

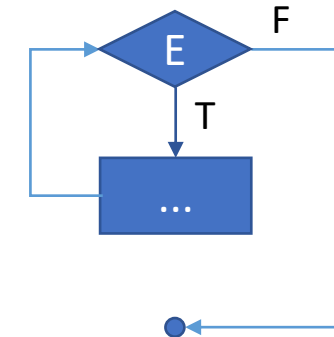
- Il *costrutto di iterazione* (ciclo)

```
while (<espressione E>) {  
    ...  
}  
...
```

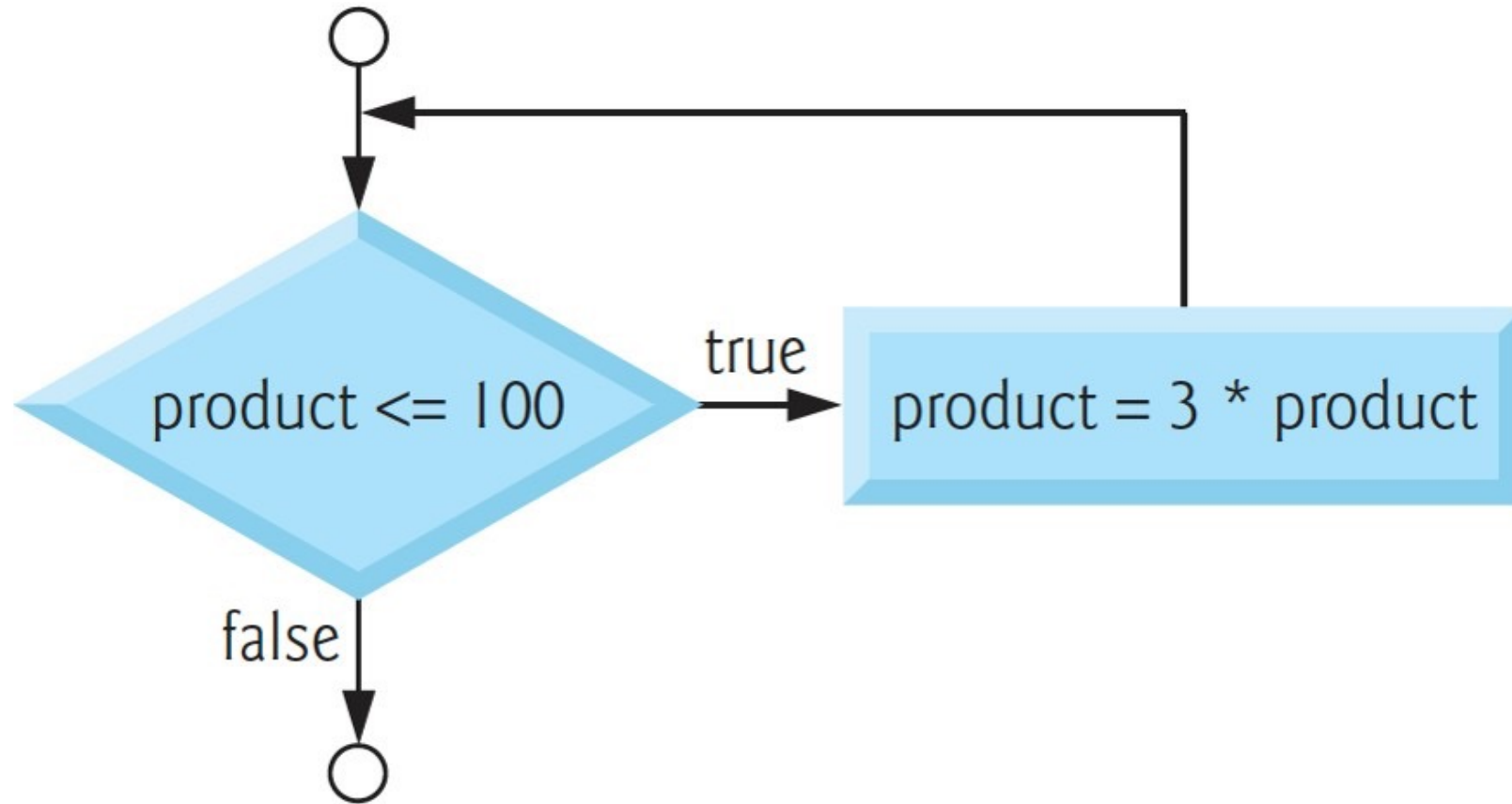


valuta l'espressione E:

- se E è vera, il blocco viene eseguito ed il controllo torna al while()
- Altrimenti, il controllo passa oltre la fine del blocco



While flowchart



Il costrutto while

- L'espressione E può essere una qualsiasi espressione:

```
char input;  
printf("Premi un tasto:");  
scanf(" %c",&input);  
while (input != 'z') {  
    printf("Sbagliato!\nPremi un tasto:");  
    scanf(" %c",&input);  
}
```

Il costrutto while

- L'espressione E, spesso, è il confronto fra una variabile indice *i* ed un valore costante

```
int i = 0;  
while (i < n) {  
    ...  
}
```

Il costrutto while

- L'indice *i* in tal caso è tipicamente aggiornato (es: incrementato di una unità) ad ogni iterazione nel corpo del ciclo

```
int i = 0;  
while (i < n) {  
    ...  
    i = i+1;  
}
```

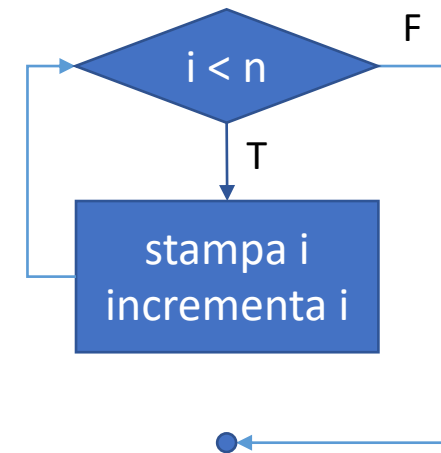
Outline

- Il costrutto while
- Esercizi
- Variabili sentinella e contatore
- Il costrutto for
- Confronto for/while

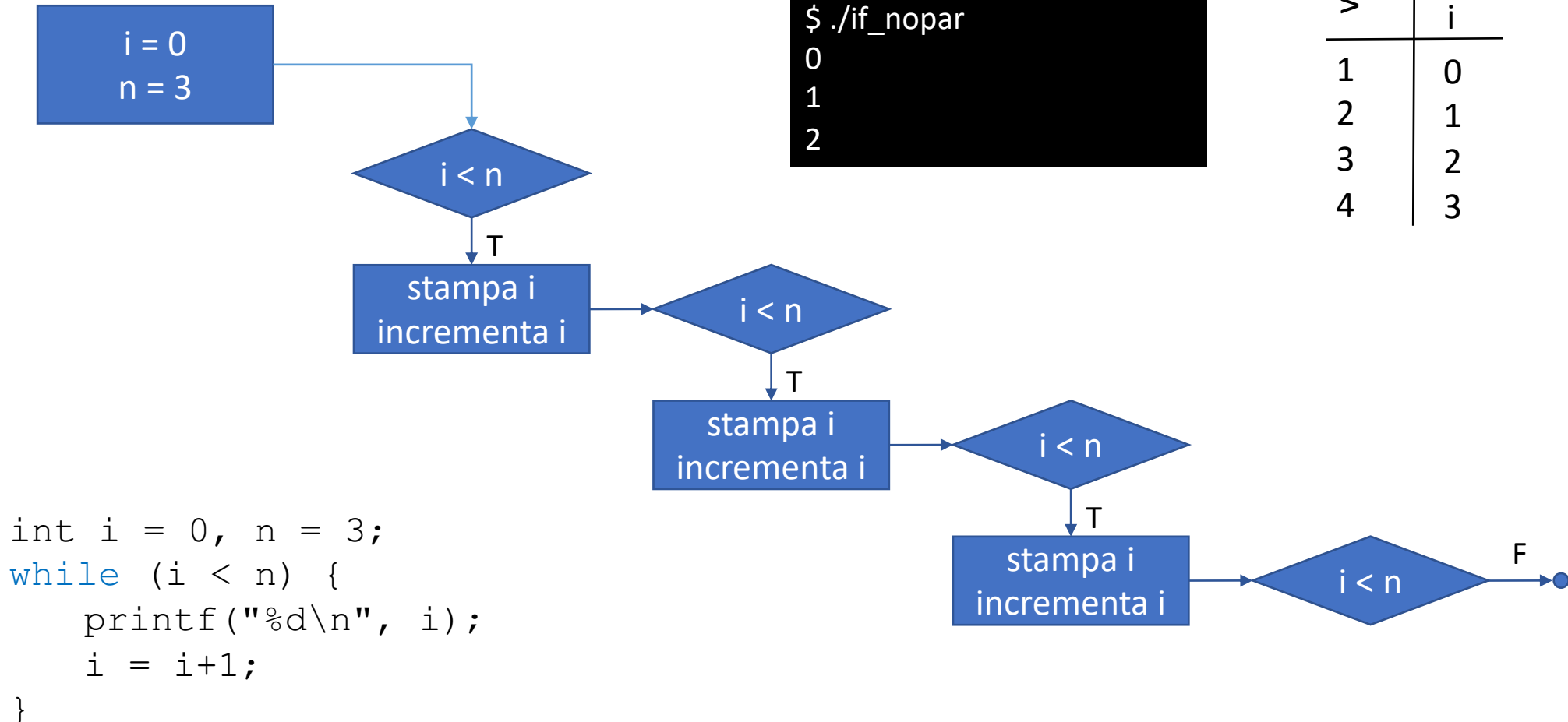
Il costrutto while - Esempio

- Stampare i primi tre numeri naturali

```
int i = 0, n = 3;  
while (i < n) {  
    printf("%d\n", i);  
    i = i+1;  
}
```



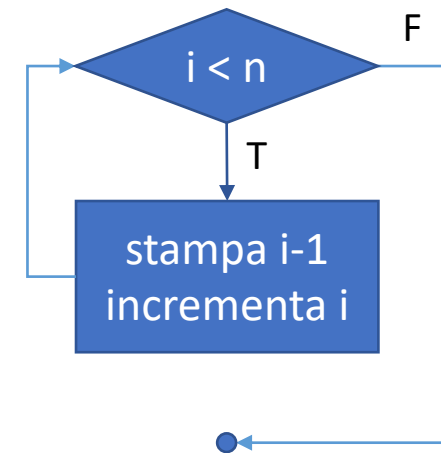
Il costrutto while - Esempio



Il costrutto while - Esempio

- Stampare i primi tre numeri naturali

```
int i = 1, n = 3;  
while (i <= n) {  
    printf("%d\n", i);  
    i = i+1;  
}
```

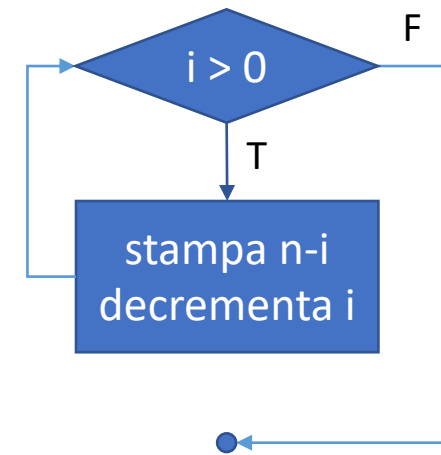


- Avremmo anche potuto inizializzare i a 1 e valutare nel *while* l'espressione $i \leq n$

Il costrutto while - Esempio

- Stampare i primi tre numeri naturali

```
int i = 3, n = 3;  
while (i > 0) {  
    printf("%d\n", n-i);  
    i = i-1;  
}
```

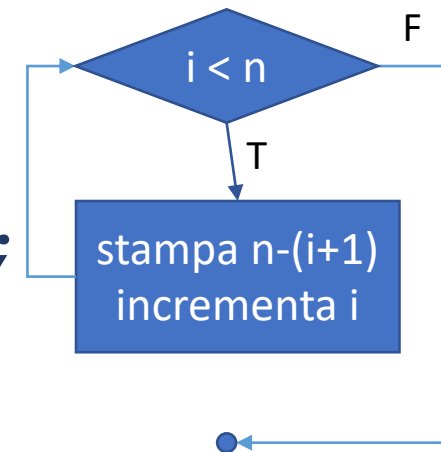


- Avremmo anche potuto decrementare i ad ogni ciclo, definendo i ed n a 3 ed aggiornando printf()

Il costrutto while - Esempio

- Stampare i primi tre numeri naturali in ordine decrescente

```
int i = 0, n = 3;  
while (i < n) {  
    printf("%d\n", n - (i+1)) ;  
    i = i+1;  
}
```

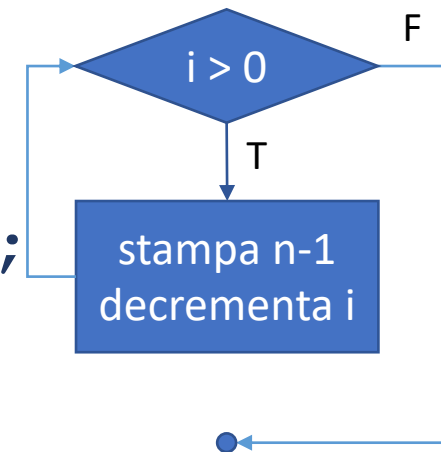


```
2  
1  
0
```

Il costrutto while - Esempio

- Stampare i primi tre numeri naturali in ordine decrescente

```
int i = 3, n = 3;  
while (i > 0) {  
    printf("%d\n", n-1);  
    i = i-1;  
}
```



```
2  
1  
0
```

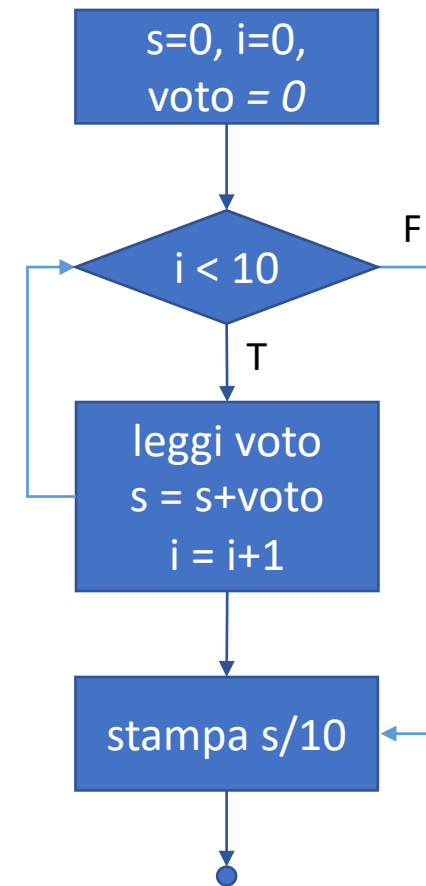
Il costrutto while

- Nell'esempio precedente l'indice di iterazione i conteneva l'informazione di interesse per l'utente
- Più comunemente, l'indice serve unicamente a contare i cicli eseguiti, mentre l'informazione di interesse per l'utente è codificata in altre variabili

Esercizio

- Calcolare la media di 10 voti inseriti dall'utente

```
int main(void) {  
    int s = 0, i = 0, voto = 0;  
    while (i < 10) {  
        printf("Inserisci voto: ");  
        scanf("%d", &voto);  
        s = s + voto;  
        i = i + 1;  
    }  
    printf("Voto medio: %f\n", s/(float)10);  
}
```



Esercizio

- Stampare m^n senza utilizzare la funzione *pow()* di *math.h*

```
int m=4, n=3, p = 1, i = 0;
while (i < n) {
    p = p * m;
    i = i + 1;
}
printf ("%d^ %d uguale %d\n", m, n, p);
```